Group 1 – Dataset Handling and Visualization

1. Create and Load different types of datasets in Python using the required libraries.

a. Creation using pandas

b. Loading CSV dataset files using Pandas

c. Loading datasets using sklearn

d. Plot

```
# IMPORT REQUIRED LIBRARIES
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets

# ----------------------------------
# a. CREATION USING PANDAS
# ----------------------------------

data = {
    'Name': ['Arun', 'Bala', 'Charan', 'Divya'],
    'Age': [20, 21, 19, 22],
    'Marks': [85, 90, 78, 88]
}

df = pd.DataFrame(data)

print("DATASET CREATED USING PANDAS")
print(df)

# ----------------------------------
# b. LOADING CSV DATASET USING PANDAS
# ----------------------------------

# Save dataframe into CSV file
df.to_csv("student.csv", index=False)

# Load CSV file
csv_data = pd.read_csv("student.csv")

print("\nCSV DATASET LOADED")
print(csv_data)

# ----------------------------------
# c. LOADING DATASET USING SKLEARN
# ----------------------------------
```

```
iris = datasets.load_iris()

iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)

print("\nIRIS DATASET USING SKLEARN")
print(iris_df.head())

# -----------------------------------
# d. PLOTTING GRAPH
# -----------------------------------

plt.plot(df['Name'], df['Marks'], marker='o')

plt.title("Student Marks")
plt.xlabel("Student Name")
plt.ylabel("Marks")

plt.grid(True)

plt.show()
```

Output:

- Dataset created using pandas
- CSV dataset loaded successfully
- Iris dataset loaded using sklearn
- Graph plotted for student marks

group 2

Group 2 – Simple Linear Regression Experiments

---

2. Imagine this dataset was collected during a medical check-up camp where weight and height of participants were recorded. Write a Python program to implement Simple Linear Regression and plot the graph.

```
import pandas as pd
import matplotlib.pyplot as plt
```

```python
from sklearn.linear_model import LinearRegression

# Dataset
weight = [[37], [40], [41], [35], [25]]
height = [152, 142, 163, 160, 171]

# Model
model = LinearRegression()
model.fit(weight, height)

# Prediction
predicted = model.predict([[32]])

print("Predicted Height for weight 32 =", predicted[0])

# Graph
plt.scatter(weight, height, color='blue')
plt.plot(weight, model.predict(weight), color='red')

plt.xlabel("Weight")
plt.ylabel("Height")
plt.title("Simple Linear Regression")

plt.show()
```

---

10. A fitness centre conducted a survey to analyze the relationship between daily exercise time (in minutes) and calories burned by participants. Construct a Python program to implement Simple Linear Regression using the above dataset. Predict the calories burned for Exercise Time = 55 minutes.

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
x = [[20], [30], [40], [50], [60], [45]]
y = [150, 200, 250, 300, 350, 280]

# Model
model = LinearRegression()
model.fit(x, y)

# Prediction
```

```
pred = model.predict([[55]])

print("Predicted Calories Burned =", pred[0])

# Graph
plt.scatter(x, y, color='blue')
plt.plot(x, model.predict(x), color='red')

plt.xlabel("Exercise Time")
plt.ylabel("Calories Burned")
plt.title("Simple Linear Regression")

plt.show()
```

---

16. A traffic analysis department wants to study how the speed of a vehicle affects the stopping distance. Based on the collected data, they want to build a model to predict stopping distance for different speeds. Write a Python program to implement Simple Linear Regression. Fit the regression model using the given dataset.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
speed = [[20], [30], [40], [50], [60]]
distance = [30, 50, 70, 90, 110]

# Model
model = LinearRegression()
model.fit(speed, distance)

# Prediction
pred = model.predict([[45]])

print("Predicted Distance =", pred[0])

# Graph
plt.scatter(speed, distance, color='blue')
plt.plot(speed, model.predict(speed), color='red')

plt.xlabel("Speed")
plt.ylabel("Distance")
```

```
plt.title("Simple Linear Regression")

plt.show()
```

---

20. A traffic analysis department wants to study how the speed of a vehicle affects the stopping distance. Based on the collected data, they want to build a model to predict stopping distance for different speeds. Write a Python program to implement Simple Linear Regression. Fit the regression model using the given dataset.

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# Dataset
speed = [[25], [35], [45], [55], [65]]
distance = [36, 56, 76, 96, 85]

# Model
model = LinearRegression()
model.fit(speed, distance)

# Prediction
pred = model.predict([[40]])

print("Predicted Distance =", pred[0])

# Graph
plt.scatter(speed, distance, color='blue')
plt.plot(speed, model.predict(speed), color='red')

plt.xlabel("Speed")
plt.ylabel("Distance")
plt.title("Simple Linear Regression")

plt.show()
```

could you solve them mathematically fro grp 2

Group 2 – Simple Linear Regression (Mathematical Solution)

The Simple Linear Regression equation is:

$$y = a + bx$$

a 1.1

b 1.0

Where:

- y = Predicted value
- x = Input value
- a = Intercept
- b = Slope

Slope Formula:

$$b = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

Intercept Formula:

$$a = \frac{\sum y - b\sum x}{n}$$

---

2. Height Prediction using Weight Dataset

Weight (x)	Height (y)	xy	x^2
37	152	5624	1369
40	142	5680	1600
41	163	6683	1681
35	160	5600	1225

Weight (x)	Height (y)	xy	x^2
25	171	4275	625

Values

- $n = 5$
- $\sum x = 178$
- $\sum y = 788$
- $\sum xy = 27862$
- $\sum x^2 = 6500$

Slope

$$b = \frac{5(27862) - (178)(788)}{5(6500) - (178)^2}$$

$$b = \frac{139310 - 140264}{32500 - 31684}$$

$$b = \frac{-954}{816}$$

$$b = -1.169$$

Intercept

$$a = \frac{788 - (-1.169)(178)}{5}$$

$$a = \frac{996.082}{5}$$

$$a = 199.216$$

Regression Equation

$$y = 199.216 - 1.169x$$

Prediction for $x = 32$

$$y = 199.216 - 1.169(32)$$

$$y = 161.808$$

Answer

Predicted Height = **161.81 cm**

---

10. Calories Burned Prediction

Exercise Time (x)	Calories (y)	xy	x^2
20	150	3000	400
30	200	6000	900
40	250	10000	1600
50	300	15000	2500
60	350	21000	3600
45	280	12600	2025

Values

- $n = 6$
- $\sum x = 245$
- $\sum y = 1530$
- $\sum xy = 67600$
- $\sum x^2 = 11025$

Slope

$$b = \frac{6(67600) - (245)(1530)}{6(11025) - (245)^2}$$

$$b = \frac{405600 - 374850}{66150 - 60025}$$

$$b = \frac{30750}{6125}$$

$$b = 5.02$$

Intercept

$$a = \frac{1530 - 5.02(245)}{6}$$

$$a = 50.17$$

Regression Equation

$$y = 50.17 + 5.02x$$

Prediction for $x = 55$

$$y = 50.17 + 5.02(55)$$

$$y = 326.27$$

Answer

Predicted Calories Burned = **326.27**

---

16. Speed vs Distance Prediction

Speed (x)	Distance (y)	xy	x^2
20	30	600	400
30	50	1500	900
40	70	2800	1600
50	90	4500	2500
60	110	6600	3600

Values

- $n = 5$
- $\sum x = 200$
- $\sum y = 350$
- $\sum xy = 16000$
- $\sum x^2 = 9000$

Slope

$$b = \frac{5(16000) - (200)(350)}{5(9000) - (200)^2}$$

$$b = \frac{80000 - 70000}{45000 - 40000}$$

$$b = \frac{10000}{5000}$$

$$b = 2$$

Intercept

$$a = \frac{350 - 2(200)}{5}$$

$$a = -10$$

Regression Equation

$$y = -10 + 2x$$

Prediction for $x = 45$

$$y = -10 + 2(45)$$

$$y = 80$$

Answer

Predicted Distance = **80 m**

---

20. Speed vs Distance Prediction (Dataset 2)

Speed (x)	Distance (y)	xy	x^2
25	36	900	625
35	56	1960	1225
45	76	3420	2025

Speed (x)	Distance (y)	xy	x^2
55	96	5280	3025
65	85	5525	4225

Values

- $n = 5$
- $\sum x = 225$
- $\sum y = 349$
- $\sum xy = 17085$
- $\sum x^2 = 11125$

Slope

$$b = \frac{5(17085) - (225)(349)}{5(11125) - (225)^2}$$

$$b = \frac{85425 - 78525}{55625 - 50625}$$

$$b = \frac{6900}{5000}$$

$$b = 1.38$$

Intercept

$$a = \frac{349 - 1.38(225)}{5}$$

$$a = 7.7$$

Regression Equation

$$y = 7.7 + 1.38x$$

Prediction for $x = 40$

$$y = 7.7 + 1.38(40)$$

$$y = 62.9$$

Answer

Predicted Distance = **62.9 m**

group 3 programs

Group 3 – K-Nearest Neighbors (KNN) Classification Experiments

---

3. A retail company wants to predict whether a customer will purchase a product (Y/N) based on Age and Salary. They use the K-Nearest Neighbors (KNN) algorithm with k = 3 to classify a new customer.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

# Dataset
data = {
    'Age': [28, 60, 35, 45],
    'Salary': [25, 110, 70, 95],
    'Purchased': ['N', 'Y', 'N', 'Y']
}

df = pd.DataFrame(data)

# Features and Target
X = df[['Age', 'Salary']]
y = df['Purchased']

# KNN Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

# New Customer
new_customer = [[52, 130]]

# Prediction
prediction = model.predict(new_customer)

print("Prediction for new customer:", prediction[0])
```

11. A company wants to automate its hiring process by predicting whether a candidate will be selected for a job. Predict whether the candidate will be Selected or Not Selected using K-Nearest Neighbors (KNN) algorithm with k = 3.

```
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

# Dataset
data = {
    'SkillsScore': [60, 90, 70, 85],
    'Experience': [1, 5, 2, 4],
    'InterviewScore': [50, 85, 65, 80],
    'Selected': ['No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Features and Target
X = df[['SkillsScore', 'Experience', 'InterviewScore']]
y = df['Selected']

# KNN Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

# New Candidate
new_candidate = [[80, 3, 75]]

# Prediction
prediction = model.predict(new_candidate)

print("Candidate Selection Prediction:", prediction[0])
```

17. A car showroom wants to predict whether a customer will buy a car. Predict whether the buyer will buy the car or

not using K-Nearest Neighbors (KNN) algorithm with k = 3. Evaluate performance metrics.

```python
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Dataset
data = {
    'Age': [25, 45, 35, 50],
    'Income': [30, 80, 50, 90],
    'CreditScore': [600, 750, 680, 800],
    'BuyCar': ['No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Features and Target
X = df[['Age', 'Income', 'CreditScore']]
y = df['BuyCar']

# KNN Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

# Prediction
new_customer = [[40, 85, 720]]
prediction = model.predict(new_customer)

print("Car Purchase Prediction:", prediction[0])

# Performance Metrics
y_pred = model.predict(X)

cm = confusion_matrix(y, y_pred)
acc = accuracy_score(y, y_pred)

print("\nConfusion Matrix:")
print(cm)

print("\nAccuracy:", acc)
```

group 4,5

Group 4 – Logistic Regression and Confusion Matrix Experiments

---

4. During a survey, individuals were classified as Owner (1) or Nonowner (0) based on Income and Lot Size. Write a Python program to implement Logistic Regression using sklearn and plot the confusion matrix.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

# Dataset
data = {
    'Income': [40, 60, 80, 100],
    'LotSize': [20, 30, 40, 50],
    'Owner': [0, 0, 1, 1]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['Income', 'LotSize']]
y = df['Owner']

# Model
model = LogisticRegression()
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Confusion Matrix
cm = confusion_matrix(y, y_pred)

print("Confusion Matrix:")
print(cm)

# Plot
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.show()
```

12. A healthcare clinic wants to develop a system to predict whether a patient has diabetes based on medical attributes. You are assigned to build a binary classification model using Logistic Regression. Compute the confusion matrix and measure their metrics.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score,

# Dataset
data = {
    'Glucose': [103, 163, 102, 112, 143, 156],
    'BMI': [24, 32, 32, 36, 42, 43],
    'Diabetes': [1, 0, 1, 0, 0, 1]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['Glucose', 'BMI']]
y = df['Diabetes']

# Model
model = LogisticRegression()
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Metrics
cm = confusion_matrix(y, y_pred)
acc = accuracy_score(y, y_pred)
prec = precision_score(y, y_pred)
rec = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

print("Confusion Matrix:")
print(cm)

print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1-Score:", f1)
```

15. A telecom company wants to predict whether a customer will leave the service or not based on Monthly Charges, Contract Duration, Customer Support Calls. Apply logistic regression model to train the model and evaluate performance metrics based on confusion matrix.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_scor

# Dataset
data = {
    'MonthlyCharges': [200, 500, 300, 450],
    'ContractDuration': [24, 6, 12, 8],
    'SupportCalls': [1, 5, 2, 4],
    'LeaveJob': ['No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Features and Target
X = df[['MonthlyCharges', 'ContractDuration', 'SupportCalls']]
y = df['LeaveJob']

# Model
model = LogisticRegression()
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Metrics
cm = confusion_matrix(y, y_pred)
acc = accuracy_score(y, y_pred)
prec = precision_score(y, y_pred, pos_label='Yes')
rec = recall_score(y, y_pred, pos_label='Yes')
f1 = f1_score(y, y_pred, pos_label='Yes')

print("Confusion Matrix:")
print(cm)

print("Accuracy:", acc)
print("Precision:", prec)
```

21. A telecom company wants to predict whether a customer will leave the service or not based on Monthly Charges, Contract Duration, Customer Support Calls. Apply logistic regression model to train the model and evaluate performance metrics based on confusion matrix.

```
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_scor

# Dataset
data = {
    'MonthlyCharges': [200, 500, 300, 450],
    'ContractDuration': [24, 6, 12, 8],
    'SupportCalls': [1, 5, 2, 4],
    'LeaveJob': ['No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Features and Target
X = df[['MonthlyCharges', 'ContractDuration', 'SupportCalls']]
y = df['LeaveJob']

# Model
model = LogisticRegression()
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Metrics
cm = confusion_matrix(y, y_pred)
acc = accuracy_score(y, y_pred)
prec = precision_score(y, y_pred, pos_label='Yes')
rec = recall_score(y, y_pred, pos_label='Yes')
f1 = f1_score(y, y_pred, pos_label='Yes')

print("Confusion Matrix:")
print(cm)

print("Accuracy:", acc)
```

```
print("Precision:", prec)
print("Recall:", rec)
print("F1-Score:", f1)
```

Group 5 – Decision Tree / ID3 Algorithm Experiments

---

5. You are working as a data analyst for a lifestyle app. Use the dataset to design a decision-making model using python code. Suggest one improvement to the dataset for better predictions.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'Weather': ['Sunny', 'Sunny', 'Windy', 'Rainy', 'Rainy', 'Rainy'],
    'Parents': ['Yes', 'No', 'Yes', 'Yes', 'No', 'Yes'],
    'Money': ['Rich', 'Rich', 'Rich', 'Poor', 'Rich', 'Poor'],
    'Decision': ['Cinema', 'Tennis', 'Cinema', 'Shopping', 'Shopping', 'Cinema']
}

df = pd.DataFrame(data)

# Label Encoding
le = LabelEncoder()

for col in df.columns:
    df[col] = le.fit_transform(df[col])

# Features and Target
X = df[['Weather', 'Parents', 'Money']]
y = df['Decision']

# Model
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# Prediction
test = [[2, 0, 0]]
pred = model.predict(test)
```

```
print("Predicted Decision:", pred)

print("\nImprovement Suggestion:")
print("Add more attributes such as Age and Location for better prediction.")
```

---

13. A company wants to automate its hiring process based on candidate profiles. Using the given dataset, build a model to predict whether a candidate should be Hired, Rejected, or Trained. Implement a Decision Tree using ID3 algorithm to predict hiring decisions based on candidate attributes.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'StudentDetails': ['MATH', 'MATH', 'CS', 'IT', 'CS', 'CS'],
    'OnlineCourses': ['Yes', 'No', 'Yes', 'No', 'No', 'Yes'],
    'Working': ['W', 'W', 'W', 'W', 'W', 'W'],
    'Decision': ['Hire', 'Reject', 'Hire', 'Train', 'Hire', 'Hire']
}

df = pd.DataFrame(data)

# Encoding
le = LabelEncoder()

for col in df.columns:
    df[col] = le.fit_transform(df[col])

# Features and Target
X = df[['StudentDetails', 'OnlineCourses', 'Working']]
y = df['Decision']

# Model
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# Prediction
test = [[1, 0, 1]]
pred = model.predict(test)
```

18. A dealership wants to automate decisions for the given dataset. Construct a program to demonstrate the working of the decision tree based ID3 algorithm.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'Income': ['High', 'Low', 'High', 'Low', 'High'],
    'FamilySize': ['Small', 'Large', 'Large', 'Small', 'Small'],
    'CreditScore': ['Good', 'Poor', 'Good', 'Good', 'Poor'],
    'BuyCar': ['Yes', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

# Encoding
le = LabelEncoder()

for col in df.columns:
    df[col] = le.fit_transform(df[col])

# Features and Target
X = df[['Income', 'FamilySize', 'CreditScore']]
y = df['BuyCar']

# Model
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# Prediction
test = [[0, 0, 0]]
pred = model.predict(test)

print("Prediction:", pred)
```

group 6,7,8

Group 6 – Recommendation System Experiments

6. An e-commerce platform wants to recommend kids dresses based on customer preferences for Colorful design, Comfort, Style, and Price. Write a python code to recommend top 2 dresses for the user.

```python
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# User Profile
users = pd.DataFrame({
    'Colorful': [5, 2],
    'Comfort': [4, 3],
    'Style': [3, 5],
    'Price': [2, 4]
}, index=['Meena', 'Kavya'])

# Dress Profile
dresses = pd.DataFrame({
    'Colorful': [5, 3, 4, 4, 2],
    'Comfort': [3, 5, 2, 5, 3],
    'Style': [5, 2, 5, 3, 2],
    'Price': [2, 4, 1, 3, 5]
}, index=['PartyFrock', 'CasualWear', 'DesignerGown', 'SummerDress', 'BudgetDress'])

# Cosine Similarity
similarity = cosine_similarity(users, dresses)

# Recommendation
for i, user in enumerate(users.index):
    scores = similarity[i]
    top2 = dresses.index[scores.argsort()[-2:][::-1]]

    print("\nRecommendations for", user)
    print(top2.tolist())
```

14. Using a content-based recommender system, compute the cosine similarity between each user and all given Tamil movies. Based on the similarity scores, determine and recommend the top 2 movies for each user.

```python
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# User Profile
users = pd.DataFrame({
    'Action': [4, 1],
    'Romance': [1, 4],
    'Comedy': [2, 3],
    'SciFi': [3, 1]
}, index=['Arun', 'Divya'])

# Movie Profile
movies = pd.DataFrame({
    'Action': [5, 1, 4, 3, 1],
    'Romance': [1, 5, 1, 1, 4],
    'Comedy': [1, 4, 3, 1, 2],
    'SciFi': [4, 1, 2, 5, 1]
}, index=['Don', 'Beast', 'Dude', '96', 'Billa'])

# Similarity
similarity = cosine_similarity(users, movies)

# Recommendation
for i, user in enumerate(users.index):
    scores = similarity[i]
    top2 = movies.index[scores.argsort()[-2:][::-1]]

    print("\nTop 2 Movies for", user)
    print(top2.tolist())
```

---

19. A Tamil online bookstore wants to recommend books to readers based on their interests in History, Romance, Drama and Adventure. Two users have given their preferences. Based on this, the system must recommend three suitable Tamil books. Write a python code for the above recommendation.

```python
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

# User Profile
users = pd.DataFrame({
    'History': [5, 1],
    'Romance': [1, 5],
    'Drama': [3, 4],
    'Adventure': [4, 2]
}, index=['Karthik', 'Ananya'])

# Book Profile
books = pd.DataFrame({
    'History': [5, 1, 4, 1, 5],
    'Romance': [2, 5, 2, 5, 3],
    'Drama': [4, 5, 3, 3, 4],
    'Adventure': [5, 1, 4, 1, 5]
}, index=[
    'PonniyinSelvan',
    'SilaNerangalilSilaManithargal',
    'ParthibanKanavu',
    'KadhalKottai',
    'SivagamiyinSabatham'
])

# Similarity
similarity = cosine_similarity(users, books)

# Recommendation
for i, user in enumerate(users.index):
    scores = similarity[i]
    top3 = books.index[scores.argsort()[-3:][::-1]]

    print("\nTop 3 Books for", user)
    print(top3.tolist())
```

Group 7 – Classification Algorithms and Performance Metrics

---

7. A hospital wants to develop an intelligent system that can predict whether a patient is at risk of heart disease based on medical parameters. Apply any one classification

algorithm and calculate accuracy, Precision, Recall and F1-score from confusion matrix.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score

# Dataset
data = {
    'BP': [120, 140, 130, 150],
    'Cholesterol': [200, 240, 220, 260],
    'HeartDisease': [0, 1, 0, 1]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['BP', 'Cholesterol']]
y = df['HeartDisease']

# Model
model = DecisionTreeClassifier()
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Metrics
cm = confusion_matrix(y, y_pred)
acc = accuracy_score(y, y_pred)
prec = precision_score(y, y_pred)
rec = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

print("Confusion Matrix:")
print(cm)

print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)
```

---

8. A hospital aims to develop an intelligent system that can predict whether a tumor is benign or malignant based

on patient medical data. Apply any one classification algorithm to predict whether the tumor is Benign and Malignant. Calculate Accuracy, Precision, Recall and F1-score from confusion matrix.

```python
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score, f1_score

# Dataset
data = {
    'TumorSize': [2, 5, 3, 6],
    'Age': [30, 50, 35, 60],
    'TumorType': [0, 1, 0, 1]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['TumorSize', 'Age']]
y = df['TumorType']

# Model
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

# Prediction
y_pred = model.predict(X)

# Metrics
cm = confusion_matrix(y, y_pred)
acc = accuracy_score(y, y_pred)
prec = precision_score(y, y_pred)
rec = recall_score(y, y_pred)
f1 = f1_score(y, y_pred)

print("Confusion Matrix:")
print(cm)

print("Accuracy:", acc)
print("Precision:", prec)
print("Recall:", rec)
print("F1 Score:", f1)
```

Group 8 – Spectrum Sensing Experiment

---

9. Implement a simulation that performs spectrum sensing using a linear classification method.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression

# Signal Data
signal_strength = np.array([[1], [2], [3], [4], [5], [6]])
channel_status = np.array([0, 0, 0, 1, 1, 1])

# Model
model = LogisticRegression()
model.fit(signal_strength, channel_status)

# Prediction
test_signal = [[4.5]]
prediction = model.predict(test_signal)

print("Predicted Channel Status:", prediction[0])

# Plot
plt.scatter(signal_strength, channel_status, color='blue')

x = np.linspace(1, 6, 100).reshape(-1,1)
y = model.predict_proba(x)[:,1]

plt.plot(x, y, color='red')

plt.xlabel("Signal Strength")
plt.ylabel("Channel Status")
plt.title("Spectrum Sensing using Linear Classification")

plt.show()
```